STATE MIND

Curve Stablecoin AI Report (draft)

2025-11-06

Table of contents

AI GENERATED

1. Project brief

Title	Description
Client	Curve
Project name	Curve Stablecoin AI Report
Timeline	2025-11-06

Project Log

curve-stablecoin

Date	Commit Hash	Note
2025-10-27	104cd635d335bd16a010ecd1820a3c48a3087934	Initial commit

snekmate

Date	Commit Hash	Note
2025-10-27	47fa27eabdc255b58b75f227866eb17220b6ffa3	Initial commit

curve-ptoracle-vy

Date	Commit Hash	Note
2025-10-27	5e0615e9af72d675ac840d2c729adfed14676027	Initial commit

Short Overview

Overview

This is an AI generated report.

2. Finding severity breakdown

All vulnerabilities discovered during the audit are classified based on their potential severity and have the following classification:

Severity	Description
Critical	Bugs leading to assets theft, fund access locking, or any other loss of funds to be transferred to any party.
High	Bugs that can trigger a contract failure. Further recovery is possible only by manual modification of the contract state or replacement.
Medium	Bugs that can break the intended contract logic or expose it to DoS attacks, but do not cause direct loss of funds.
Informational	Bugs that do not have a significant immediate impact and could be easily fixed.

Based on the feedback received from the Client regarding the list of findings discovered by the Contractor, they are assigned the following statuses:

Status	Description
Fixed	Recommended fixes have been made to the project code and no longer affect its security.
Acknowledged	The Client is aware of the finding. Recommendations for the finding are planned to be resolved in the future.

3. Summary of findings

Severity	# of Findings
Critical	0 (0 fixed, 0 acknowledged)
High	2 (1 fixed, 0 acknowledged)
Medium	5 (0 fixed, 0 acknowledged)
Informational	10 (0 fixed, 0 acknowledged)
Total	17 (1 fixed, 0 acknowledged)

4. Conclusion

5. Findings report

HIGH-01	Incorrect _update_discount_params validation	New

Description

_update_discount_params validates using the current self.slope/self.intercept instead of the proposed _slope/_intercept, so unsafe new values can be accepted and later cause reverts when used at runtime.
Impact:

- Protocol-wide DoS: If discount > DISCOUNT_PRECISION, discount_factor underflows and reverts under checked arithmetic, breaking price() and price_w() across the system.

Lines: **PtOracle.vy PtOracle.vy:243-247**

Recommendation

Enforce the invariant 0 <= discount < DISCOUNT_PRECISION end-to-end and document it. _update_discount_params: validate the proposed values, not the stored ones. Recompute time_to_maturity_years from current state and compute new_discount using _slope and _intercept, then assert new_discount < DISCOUNT_PRECISION before updating storage. Include clear revert messages. Optionally cap acceptable discounts further (e.g., <= 95% of DISCOUNT_PRECISION) to reduce risk from extreme settings.

HIGH-02	Incorrect Padding Detection in Base64 Decoding	Fixed at: b58155f

Description

The **_decode** function in **base64.vy** has incorrect padding detection logic. The code checks **c3 == 63** to detect double padding (**==**), but index 63 corresponds to the non-padding characters **/** (standard alphabet) or **_** (URL-safe alphabet), not the padding character. The padding character **=** is at index 64 in both alphabets. As a result, valid inputs with **/** (or **_**) as the third character of a 4-byte chunk are incorrectly treated as if they had **==** padding and truncated. In addition, the order of the padding checks is wrong: the **if c4 == 64** branch (single **=** padding) is evaluated before the **elif c3 == 63** branch (intended for **==**). Even if the index were corrected to **64**, any properly padded **==** chunk would still match **c4 == 64** first and be handled as a single **=** case rather than as double padding. Together, these issues mean that both double-padded Base64 strings and certain unpadded strings are decoded incorrectly.
Impact:
Base64 strings with double padding (**==**) will not be correctly decoded, and some valid Base64 strings with **/** (standard) or **_** (URL-safe) in the third position (e.g. **"AA/A"**) will be truncated. This leads to incorrect output data and silent data corruption in applications relying on this module for Base64 decoding, which can compromise data integrity and any higher-level logic that depends on correct decoding.
Lines: **base64.vy 224, 261**

Recommendation

Fix both the constant and the branch ordering. First, change the constant from **elif c3 == 63:** to **elif c3 == 64:** so that the third character is correctly compared against the padding character **=**. Second, reorder the checks so that the double-padding case is handled before the single-padding case, for example:

- If **c3 == 64** and **c4 == 64**, treat the chunk as **==** and only emit the first byte.
- Else if **c4 == 64** and **c3 != 64**, treat the chunk as **=** and emit the first two bytes.
- Else, emit all three bytes.

Optionally, add assertions to reject invalid padding combinations (e.g. **c3 == 64** while **c4 != 64**).

MEDIUM-01	Immediate parameter updates and weak rate limiting in PtOracle enable MEV exploitation	New

Description

PtOracle's pricing parameters are updated via set_linear_discount and set_slope_from_apy and take effect immediately. The update gating uses a strict inequality check block.timestamp > last_discount_update + min_update_interval. Because min_update_interval can be configured to 0 and there is no timelock or commit-reveal, updates can be executed as frequently as once per block/second and applied atomically. MEV exposure: bots can monitor the mempool for parameter update transactions, front-run with trades at the old price, let the update execute, and then back-run at the new price to capture the discrete price jump; rapid toggling exacerbates this.

Impact:

- MEV/sandwich risk: Attackers can front-run on old parameters and back-run on new parameters to arbitrage the discrete price change, extracting value from the protocol and its users.
- Volatility and coordination risk: Managers can rapidly flip parameters (especially with min_update_interval = 0), creating unpredictable pricing and harming integrators that assume stability.
- Reputational and financial impact: Predictable, profitable MEV opportunities can degrade user trust and lead to material losses over time.

Lines: **PtOracle.vy**

PtOracle.vy:262-270, PtOracle.vy:274-305, PtOracle.vy:265-269, PtOracle.vy:282-285, PtOracle.vy:267, PtOracle.vy:284, PtOracle.vy:318-320

Recommendation

- Add delay/commit mechanisms for sensitive parameter changes: implement a two-step commit-reveal scheme or a timelock so changes cannot be immediately exploited. Consider batching updates at fixed intervals and requiring multi-signature approval with a time delay.
- Enforce sensible lower bounds and cooldowns: set a non-zero minimum for min_update_interval, add per-parameter cooldowns, and cap maximum change per interval to prevent rapid flip-flopping.
- Correct the rate-limit predicate: change the comparison to block.timestamp >= last_discount_update + min_update_interval so the effective interval matches the configured value and updates are allowed exactly at T + N. If the current strict behavior is intentional, document it clearly and consider renaming the variable (e.g., min_seconds_before_next_update).
- Reduce discrete jumps: introduce gradual/ramped transitions so parameter changes phase in over time rather than taking effect instantly, reducing arbitrageable discontinuities.
- Execution privacy: when executing updates, prefer private transaction channels (e.g., Flashbots) to reduce mempool signaling and front-running risk.
- Process and documentation: publicly announce scheduled changes where appropriate, document off-chain procedures, and align integrator expectations with on-chain behavior.

MEDIUM-02	View Function users_to_liquidate Returns Positions That Will Fail Liquidation	New

Description

The users_to_liquidate view function (lines 50-82) returns positions where health < HEALTH_THRESHOLD and the user has approved the zap, but it doesn't filter out positions that would fail the **ratio > WAD** check (line 109 in liquidate_partial). A position is included in the results at lines 72-81 even if the calculated ratio (line 71) is <= WAD. Liquidation bots consuming this view function will attempt to liquidate these positions, only to have their transactions revert with 'position rekt' at execution time. This wastes gas and reduces the utility of the view function for liquidation discovery.

Impact:

Liquidation bots and users calling users_to_liquidate receive misleading data, including positions that cannot actually be liquidated through this zap. This causes: (1) Wasted gas when bots attempt to liquidate positions that will revert. (2) Reduced efficiency of liquidation discovery requiring bots to simulate each liquidation off-chain. (3) Poor user experience as the view function doesn't accurately represent liquidatable positions. (4) Potential competitive disadvantage for liquidators using this view function versus direct Controller access. While not causing fund loss, it significantly degrades the zap's practical utility.

Lines: **PartialRepayZap.vy 50-82**

Recommendation

Add a check in the loop before appending positions: **if ratio <= WAD: continue** after line 71 to skip positions with insufficient collateral ratio. Change to:

```
ratio: uint256 = unsafe_div(unsafe_mul(x_down, WAD), pos.debt)
if ratio <= WAD:
    continue  # Skip positions that would fail ratio check
out.append(...)
```

This ensures only actually liquidatable positions are returned, improving the view function's reliability and reducing wasted gas for liquidators.

MEDIUM-03	price_w timestamp-only cache not invalidated on same-block parameter updates, causing stale prices vs price()	New

Description

price_w caches last_price and returns it whenever block.timestamp == last_update (timestamp-based cache gate). Parameter update functions (set_linear_discount, set_slope_from_apy) modify pricing inputs (e.g., slope, intercept) and last_discount_update, but they do not touch the cache variables last_price/last_update. As a result, if price_w is called and caches a value earlier in a block, and then pricing parameters are updated later in that same block, subsequent price_w calls still return the stale cached value, while price() recomputes and reflects the new parameters. This creates a within-block inconsistency between price_w and price() and effectively delays parameter changes for consumers of price_w until the next block. The root cause is a cache invalidation gap: the cache key only considers timestamp and not whether dependent parameters changed.

Impact:

- Within the same block after a parameter update, price_w returns a stale cached price until the next block, while price() returns the updated price.
- Integrators relying on different functions (price_w vs price()) can observe divergent prices in the same block, leading to inconsistent behavior and integration complexity.
- Potential MEV/arbitrage opportunities arise from temporary price discrepancies across protocols or venues that consume different price interfaces.
- Incorrect or inconsistent valuations in DeFi protocols during the affected block.
- Unexpected delay in governance/parameter change effectiveness for consumers of price_w.
- Behavior is unexpected and undocumented, increasing the risk of misuse by integrators.

Lines: **PtOracle.vy PtOracle.vy:197-205, PtOracle.vy:249-256, PtOracle.vy:262-270, PtOracle.vy:275-306**

Recommendation

Adopt one of the following (prefer 1 for correctness + gas efficiency):

1. Versioned cache key: Maintain a parameter_version (or discount_params_version) incremented in the parameter update path (e.g., inside _update_discount_params). Store cached_version alongside last_price/last_update. In price_w, return the cache only if both block.timestamp == last_update AND cached_version == parameter_version; otherwise recompute and update last_price, last_update, and cached_version.
2. Timestamp-based invalidation: Incorporate last_discount_update into the cache condition. In price_w, if last_discount_update >= last_update, recompute even when block.timestamp == last_update.
3. Immediate invalidation on updates: In _update_discount_params (or in set_linear_discount/set_slope_from_apy), invalidate the cache by resetting last_update (e.g., to 0) and/or clearing a cache-valid flag so the next price_w call recomputes.
4. If caching is non-critical, remove the cache in price_w to guarantee consistency with price() across all calls.

Additionally, document the chosen behavior clearly so integrators understand any within-block nuances.

MEDIUM-04	Liquidation Discount Not Updated For Third-Party Managed Positions	New

Description

In Controller.vy _add_collateral_borrow function (lines 766-770), the liquidation_discount is updated to the current global value only when **_for == msg.sender** (line 766). When someone else manages the position (with approval), the old liquidation_discount is kept (line 770). This means positions managed by third parties retain stale liquidation_discounts even after global parameters are updated. If the admin decreases liquidation_discount to make liquidations safer for borrowers, third-party managed positions don't get the benefit. Conversely, if increased for safety, these positions remain at old (possibly dangerous) levels.

Impact:

Stale liquidation discounts cause:

1. Positions managed by protocols, bots, or delegated managers don't get parameter updates,
2. Inconsistent treatment between self-managed and third-party managed positions,
3. Protocol unable to uniformly update risk parameters across all positions,
4. Some users subject to harsher/more lenient liquidation than intended by current parameters,
5. Governance parameter changes don't fully take effect across the system. This creates unfairness and reduces admin's ability to manage protocol risk uniformly.

Lines: **Controller.vy 766-770**

Recommendation

Update discounts consistently:

1. Always update liquidation_discount to current global value regardless of who calls the function,
2. If maintaining old discounts is desired, document the rationale clearly,
3. Add function allowing users to manually sync their liquidation_discount to current value,
4. Consider deprecating the per-user liquidation_discount in favor of global-only,
5. Add monitoring to track positions with stale parameters. Provide view function showing which positions have outdated discounts.

MEDIUM-05	borrowed_balance calculation can underflow and revert (view DoS and UX breakage)	New

Description

In LendController.borrowed_balance the expression **balance = VAULT.asset_balance() + core.repaid − core.lent − self.collected** relies on checked arithmetic. If **VAULT.asset_balance() + repaid** is less than **lent + collected**, the subtraction underflows and reverts, even though the function intends to be a non-mutating getter. There is a TODO in comments about handling underflow, but it is not implemented. A revert here can break **maxWithdraw**, **previewWithdraw**, and other ERC4626 views that depend on **borrowed_balance**, degrading UX or causing downstream assumptions to fail.

Impact:

Read-only DoS of vault/accounting views when the controller has no idle cash (or accounting becomes temporarily imbalanced), causing frontends and integrators to fail on view calls and preventing users from getting previews or limits.

Lines: **contracts/lending/LendController.vy 164-171**

Recommendation

Use saturating arithmetic for the entire idle-cash calculation. For example:

- Compute **positive = VAULT.asset_balance() + core.repaid**
- Compute **negative = core.lent + self.collected**
- **balance = crv_math.sub_or_zero(positive, negative)**
- Then apply **balance = crv_math.sub_or_zero(balance, core.admin_fees)** to avoid any underflow-driven revert while reflecting zero when negative. Add tests around boundary conditions to ensure getters never revert.

INFORMATIONAL-01	Incorrect function selector used when reading gauge type	New

Description

StableswapFactoryHandler._get_gauge_type performs a raw_call using the selector for gauge_type(address), but the correct method is gauge_types(address) as declared in the interface. This mismatch will make the call fail, returning zero gauge type and causing misclassification of gauges.

Impact:

Incorrect gauge type detection can break tooling or logic relying on gauge classification, potentially affecting incentive routing and analytics.

Lines: **StableswapFactoryHandler.vy 199–206**

Recommendation

Update the raw_call to use method_id("gauge_types(address)") to match the actual GaugeController ABI, or call the interface's typed view directly to avoid selector mismatches.

INFORMATIONAL-02	Vault Share Rounding Can Steal User Deposits	New

Description

In Vault.vy _convert_to_shares function (lines 161-172), share calculation uses integer division that can round down to zero. If a user deposits a small amount of assets when totalSupply is large, the calculation **numerator // denominator** at line 170 could evaluate to 0 shares. However, the assets are still transferred from the user (Vault.vy line 271). This means user pays assets but receives 0 shares, permanently losing their funds. The MIN_ASSETS check (line 268) validates total vault assets after deposit, but doesn't prevent individual deposits from minting 0 shares. For example, if totalSupply is 1e24 and total_assets is 1e24, depositing 1 wei would mint 0 shares but still transfer the 1 wei.

Impact:

Users lose funds through 0-share deposits:

1. Small deposits permanently lost as shares round to 0,
2. Attackers can intentionally inflate totalSupply to make subsequent deposits round to 0,
3. First depositors after inflation attacks lose funds,
4. No way for users to recover lost assets,
5. Breaks fundamental vault invariant that deposits should give proportional shares. Even small per-deposit losses accumulate to significant amounts over many transactions.

Lines: **Vault.vy 161–172, 260–276**

Recommendation

Prevent zero-share mints:

1. Add explicit check **assert to_mint > 0** before minting shares in deposit function,
2. Implement minimum share amount per deposit (e.g., 1000 shares minimum),
3. Add virtual share offset larger than DEAD_SHARES to prevent inflation attacks,
4. Revert deposit if shares would round to 0 instead of accepting assets,
5. Consider ERC4626 inflation attack mitigations from OpenZeppelin. Add fuzz tests for edge cases with extreme totalSupply/totalAssets ratios.

INFORMATIONAL-03	Factory Can Be Deployed With Zero Fee Receiver Breaking Fee Collection	New

Description

In LendingFactory.vy constructor (lines 68-94), the _fee_receiver parameter is accepted without validation that it's not empty(address). The fee_receiver is stored at line 94 without any checks. However, set_fee_receiver function (lines 270-278) does validate this with assert at line 276. If the factory is mistakenly deployed with empty(address) as fee_receiver, then all calls to collect_fees in controllers would attempt to transfer tokens to address(0). Most ERC20 tokens reject transfers to zero address, causing collect_fees to permanently revert. This would prevent admin fee collection across all markets until fee_receiver is updated.

Impact:

Zero address fee receiver causes:

1. Complete failure of fee collection mechanism across all markets,
2. Admin fees accumulate but cannot be collected until fee_receiver is updated,
3. During the period of accumulation, fees are locked and potentially at risk,
4. Requires governance action to fix, causing delays in fee collection,
5. If factory deployment is immutable or upgrading is difficult, this could be a permanent issue. This breaks a core economic mechanism of the protocol.

Lines: **LendingFactory.vy 68-94**

Recommendation

Add validation in factory constructor:

1. Assert that _fee_receiver != empty(address) in init,
2. Match the validation pattern used in set_fee_receiver for consistency,
3. Consider also validating other constructor parameters like blueprints and admin,
4. Add deployment checklist that verifies all parameters before mainnet deployment,
5. Include factory initialization in integration tests.

INFORMATIONAL-04	Misleading 0 = no limit comment for max_slope_change/max_intercept_change	New

Description

The comment on max_slope_change and max_intercept_change says **0 = no limit**, but the implementation uses **change <= limit** and the constructor initializes both limits to max_value(uint256) as the real "no limit" sentinel. With this logic, a limit of 0 actually forbids any non-zero change (hard freeze), while max_value(uint256) behaves as unbounded.

Impact:

- Admins following the comment may set a limit to 0 expecting "no limit", but instead freeze discount updates until limits are raised again.
- This is a misconfiguration/UX risk, not a direct exploit; impact is temporary DoS of parameter updates and operational confusion.

Lines: **PtOracle.vy PtOracle.vy:45-49, PtOracle.vy:124-129, PtOracle.vy:219-235**

Recommendation

Update comments to match behavior (e.g., **max_uint = no limit; 0 = no change allowed**) and/or adjust the code so that **0** truly means "no limit" by skipping the check when the limit is 0. Add tests and docs that explicitly cover **0**, bounded values, and **max_value(uint256)** semantics.

INFORMATIONAL-05	Chainlink-based oracles do not validate staleness of answers	New

Description

Some oracle wrappers read Chainlink answers but do not check the **updatedAt** timestamp or **answeredInRound**, nor do they impose a staleness threshold. For example, GoldChainlinkPrice._price() only checks **price > 0**. Similarly, ChainlinkWrapper.rate() returns **latestRoundData().answer** without staleness checks. If the feed stalls or is stale while still returning a nonzero answer, consumers may use outdated prices.
Impact:
Use of stale or invalid prices can lead to mispricing, incorrect risk decisions, or economic losses in dependent systems.
Lines: **contracts/price_oracles/GoldChainlinkPrice.vy 33–39**

Recommendation

Enforce staleness checks similar to other oracles in this repo that verify **block.timestamp – updatedAt <= threshold** and **answeredInRound >= roundID**. Apply to GoldChainlinkPrice.vy (_price), ChainlinkWrapper.vy (rate), and any other Chainlink wrappers lacking freshness validation.

INFORMATIONAL-06	Unvalidated Controller Parameter Enables Malicious Controller/Token Injection: Unlimited Approvals, Token Theft, Bot Data Poisoning, and Event Pollution	New

Description

Pattern: Multiple functions in PartialRepayZap accept a user-supplied _controller (IController) address with no validation. The zap then trusts data and token addresses returned by that controller, grants approvals based on them, performs external calls, and even emits the unvalidated address in events. At line 103, the contract calls **tkn.max_approve(BORROWED, CONTROLLER.address)**, which approves the maximum uint256 value of borrowed tokens to the controller. The max_approve function in token_lib.vy (lines 6-8) only sets approval if current allowance is 0, but once set, the approval remains at max_value(uint256) indefinitely. If the controller contract has any vulnerabilities or becomes compromised, all borrowed tokens held by the zap can be drained. This violates the principle of least privilege.
Impact:
If the controller is compromised or has an exploitable vulnerability, an attacker could drain all borrowed tokens from the zap contract. While the zap is not designed to hold tokens long-term, tokens could be present during transactions or if accidentally sent to the contract. The unlimited approval creates unnecessary systemic risk.
Lines: **PartialRepayZap.vy 103**

Recommendation

Mitigate by eliminating trust in user-supplied controllers and minimizing approval. Instead of unlimited approval, calculate the exact amount needed and approve only that amount. Change line 103 to approve **borrowed_from_sender** after calculating it at line 113. Use:
assert extcall BORROWED.approve(CONTROLLER.address, borrowed_from_sender, default_return_value=True). Reset approval to 0 after the liquidate call completes, or use the approve-transfer-revoke pattern.

INFORMATIONAL-07	Extra Health Allows DOS	New

Description

In Controller.vy set_extra_health function (lines 1488-1495), users can set extra_health[msg.sender] to any uint256 value without bounds. This value is added to loan_discount in calculations like _calculate_debt_n1 (line 497) and max_borrowable (line 313). While the intention is to allow users to create safer positions, there's no maximum limit. A user could set an extremely high extra_health value to set it to max_value(uint256) to cause overflow in the addition **self.loan_discount + self.extra_health[user]**.

Impact:

Unbounded extra_health manipulation leads to overflow in loan_discount calculations causing DOS

Lines: **Controller.vy 1488–1495, 497, 313**

Recommendation

Add bounds to extra_health - implement maximum extra_health value (e.g., 10% of loan_discount)

INFORMATIONAL-08	Plain pool deployment does not enforce intended fee bounds (0.04%–1%); only checks fee > 0	New

Description

StableswapFactory.deploy_plain_pool only asserts _fee > 0. The docstring suggests an expected range of 0.04% to 1%, but these bounds are not enforced on-chain. Because pool deployment is permissionless, anyone can deploy a pool with excessive or microscopic fees, potentially confusing integrators and users.

Impact:

Misconfigured pools with extreme fees may be deployed, degrading user experience or enabling fee-gouging. While not an exploit against the factory, this is a configuration risk that may affect ecosystem safety if tooling assumes reasonable fee ranges.

Lines: **StableswapFactory.vy 647-653**

Recommendation

Enforce explicit fee bounds to match documentation (e.g., assert 4_000_000 <= _fee <= 100_000_000 with 1e10 precision), or clearly document that the factory is permissionless and does not guarantee parameters. Tooling should flag abnormal fees.

INFORMATIONAL-09	Constructor does not enforce borrowing discount invariants (loan_discount > liquidation_discount, bounds)	New

Description

The Controller constructor assigns loan_discount and liquidation_discount directly without validating their relationship or bounds. Later, set_borrowing_discounts enforces invariants (loan_discount > liquidation_discount, liquidation_discount >= MIN_LIQUIDATION_DISCOUNT, loan_discount <= MAX_LOAN_DISCOUNT), but deployments can start with invalid values if misconfigured. Incorrect discounts can break health/borrowing assumptions, cause unexpected reverts in borrowing/liquidation paths, or permit excessively risky parameters until corrected.

Impact:

- Markets may deploy with unsafe or inconsistent liquidation/borrowing thresholds, potentially enabling over-borrowing or preventing liquidations.
- Subsequent operations depending on these parameters may revert or behave unpredictably until governance corrects them.

Lines: **Controller.vy 140–185, 1408–1425**

Recommendation

Validate on construction: assert loan_discount > liquidation_discount, liquidation_discount >= MIN_LIQUIDATION_DISCOUNT, and loan_discount <= MAX_LOAN_DISCOUNT. Mirror the same checks performed in set_borrowing_discounts.

INFORMATIONAL-10	MintController passes empty view implementation to core initializer (deployment footgun)	New

Description

MintController.init forwards **empty(address)** as the view implementation to **core.__init__**, but the core initializer immediately asserts the view address is non-zero. Unless this parameter is substituted during blueprint instantiation, deployment will revert. The code comment indicates an external replacement step at deployment, but the guard in core enforces non-empty on construction.

Impact:

Misconfigured deployments will fail during initialization. This is a deploy-time hazard that can be overlooked and lead to failed markets or wasted gas.

Lines: **contracts/MintController.vy 32–40**

Recommendation

Ensure the blueprint deployment mechanism always passes a valid **view_impl** to **core.__init__**. Alternatively, refactor MintController to call **core.__init__** only after a valid view address is available, or accept empty at init and require an immediate admin **set_view** with a non-empty address before enabling the market.


STATE
MIND